

Actividad Practica N° 2
Sistemas Operativos Avanzados
Año: 2025
2°Cuatrimestre

Fecha de entrega límite	
Parte 1 (Actividad 2)	• 20 de Octubre (Comisión lunes) • 21 de Octubre (Comisión martes)
Parte 2 (Actividad Integrador)	• 17 de Noviembre (Comisión Lunes) • 18 de Noviembre (Comisión martes)

Objetivo

El objetivo de este trabajo práctico consiste en realizar el pasaje del circuito simulado a un circuito físico y, a su vez, su comunicación con un dispositivo móvil que posea el S.O Android. La forma de conectividad entre el sistema embebido (ESP32) y el Smartphone dependerá del tipo de placa desarrollo empleada. Por eso a continuación se deberá utilizar Wifi como mecanismo de comunicación.

Para ello este Trabajo se dividirá en 2 Partes que se describen en las siguientes páginas.

Parte 1	2
Parte2	3
Uso de biblioteca Metrics.h	4

Parte 1



En la Parte 1 de este trabajo práctico se deberá realizar lo siguiente:

Usar el repositorio de github asignado a cada grupo por la catedra.

- **EMBEBIDO:** Aquí se deberá colocar el archivo con extensión. INO y las bibliotecas utilizadas en el Código del ESP32 Físico.
 - **Proyecto_sin_vibecoding:** Directorio con el código del proyecto desarrollado de forma manual. Sin utilizar la técnica de vibecoding
 - **Proyecto_con_vibecoding:** Directorio con el código del proyecto desarrollado utilizando la técnica de vibecoding
- **ANDROID:** Aquí se deberá colocar los archivos del proyecto de la aplicación de Android Studio, sin ningún archivo binario.
- **INFORMES:** Aquí se deberán colocar los documentos de los informes generados en el proyecto

Circuito ESP32 Físico.

1. Pasar el circuito simulado al circuito físico, utilizando para la placa de prototipado elegida. Además, respetando la cantidad y tipo de sensores y actuadores presentados en el TP1
2. Utilizar una fuente externa (cargador, power bank, etc) en lugar de la utilizada en el simulador

Código del ESP32 Físico

1. Adaptar la máquina de estados desarrollada para que:
 - Cuando se reciba un comando por Wifi, realice una acción en el sistema embebido.
 - En un determinado momento informar por Wifi el valor de un sensor al Smartphone
2. Realizar las pruebas de conectividad entre Android y Esp32 con una aplicación cliente descargada de la *Play Store* del dispositivo móvil.
3. Implementar la biblioteca Metrics.h, su utilización se explica al final del documento.

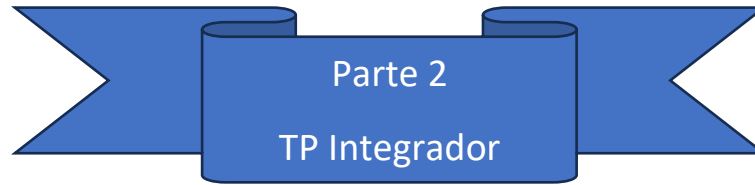
Código Android

Empezar a desarrollar una aplicación para Android que involucre los diferentes conceptos vistos en clase, utilizando el IDE de Android Studio y en lenguaje Java/Kotlin.

1. Desarrollar en Android Studio, el diseño de la aplicación Android, que se integrará a la lógica del sistema físico.
2. La aplicación de Android debe ser acorde a la temática del TP1.
3. Hacer como mínimo dos Activities.
 - a. **Primer Activity:** Tiene que haber una pantalla esquemática o de prototipo con un botón y un campo de texto como mínimo. Cuya lógica será completada en el TP de integración (Parte 2).
 - b. **Segunda Activity:** Tendrá todo lo necesario para interactuar con un sensor del dispositivo móvil. Para que realice alguna acción ya sea en el smartphone o en el sistema embebido

FORMA DE ENTREGA (TP2 Parte1)

1. El circuito físico, junto con la conexión con el cliente Android descargado de la Playstore, deberá ser mostrado en clase a los docentes en la fecha de entrega pautada.



Código de Android

1. Completar la lógica de la primera Activity para que haga lo siguiente:
 - Realizar el envío del comando de Wifi, desarrollado en la parte 1, al embebido físico para que realice una acción.
 - Mostrar por pantalla el valor de algún sensor, que sea enviado desde embebido físico a través de wifi.

INFORME INTEGRADOR

Desarrollar el informe en formato paper. Agregando las secciones de encabezado, introducción, desarrollo, conclusiones y bibliografía (en formato IEEE), formato CACIC. El formato paper solicitado se muestra en el siguiente enlace:

https://www.dropbox.com/scl/fi/bpp2ypzsuk640fjyo7kql/00_EstructuraPaper_cacic.doc?rlkey=0rpybimvlzu88i565ijxbq8m&dl=0

El contenido que deberán tener dichas secciones se detalla a continuación:

En el encabezado:

1. Debe indicarse el nombre de la aplicación como título del Paper.
2. Indicar Nombres, Apellido y DNI de cada integrante del grupo. Así cómo también debe indicarse el día de cursada y el número de grupo.
3. Agregar un resumen del trabajo realizado, de hasta 150 palabras como máximo.

En Introducción:

1. Describir la funcionalidad de la aplicación. En este punto se debe comentar cual es utilidad de la aplicación y que es lo que hace.

En el desarrollo:

1. Deberán indicar el enlace al repositorio GitHub y de Wokwi
2. Agregar el diagrama de máquina de estados generado en la actividad1
3. Agregar una captura de pantalla del circuito generado wokwi
4. Deberá contener un diagrama funcional/navegación de las Activities. En otras palabras, mostrar gráficamente, que Activities llama a otra Activity.
5. Escribir un manual de usuario, en donde se describa como se utiliza la aplicación y como interactúa con el sistema embebido.

Bibliografía:

1. La bibliografía debe ser referenciada y estructurada en formato IEEE

FORMA DE ENTREGA (Parte 2 TP Integrador):

- El circuito físico con comunicación a la Aplicación Android desarrollada deberá ser mostrada en el laboratorio en la fecha de entrega pactada, la cual fue mencionada anteriormente.
- Se deberá entregar el Informe en la plataforma de Miel, en la sección de práctica.

Uso de biblioteca Metrics.h

Objetivo: Medir y promediar el uso de CPU (por core) y de memoria (heap) en un ESP32 utilizando la biblioteca provista y documentar resultados en dos situaciones distintas.

Uso de la biblioteca:

1. Incluir en el código la biblioteca para obtener la métrica de CPU y memoria que se encuentra en el siguiente repositorio

<https://gitlab.com/so-unlam/Material-SOA/-/tree/master/Ejemplos%20SE/UsoCpuMemESP32>

Utilizarla de la siguiente manera:

```
#include "Metrics.h"

void setup()
{
    .....
    // inicializar las mediciones de muestreo
    initStats();
}

void loop()
{
    .....
    //En alguna parte del código ejecutar una sola vez la siguiente funcion
    finishStats();
}
```

2. En el setup llamar a initStats() para empezar a realizar el muestreo de las mediciones de cpu y memoria
3. Cuando quieran cerrar el período de medición y obtener promedios: llamar a finishStats(); debe imprimirse en el Monitor Serie algo como lo siguiente.

```
=== Promedio del intervalo (initStats() → finishStats()) ===
Muestras: 11 (* 11 s)

=== Contribución al total del sistema (promedio) ===
Core 0 -> Total: 100.00% | Ocupado: 0.01% | Libre (IDLE): 99.99%
Core 1 -> Total: 100.00% | Ocupado: 0.22% | Libre (IDLE): 99.78%

=== Estado de la memoria en ESP32 (promedio) ===
=== Memoria interna (Heap) ===
Heap total : 378828 bytes
Heap libre : 325156 bytes
Heap usado : 53672 bytes
Uso        : 14.17 %
```

4. En la entrega del Integrador se deberá presentar un archivo Word, llamado ***“metricasCPU”***, en el que se incluya la impresión del promedio de uso de CPU y de memoria obtenidos mediante la función finishStats(), tal como se muestra en el monitor serial.
5. Estas mediciones deberán realizarse en dos situaciones específicas

A. Reposo del sistema: En loop(), tras 10 segundos sin interacción del usuario, llamar finishStats() y registrar la impresión del monitor serie.

B. Tras una acción específica: Repetir el proceso pero disparando una acción concreta (ejemplos: lectura de un sensor, recepción por MQTT desde el broker). La acción debe ocurrir dentro de la ventana de medición (entre initStats() y finishStats()).

6. El archivo Word debe tener el resultado de las situaciones realizadas tanto con el código sin vibecoding, como con el realizado con vibecoding.